```
var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
builder.Services.AddRazorPages();
```

In addition to the framework's use of its container, you are encouraged to use it to register your own services, so they can be injected into consumers by the container, which will take responsibility for managing the lifetime (creation and destruction) of those services on your behalf. A key point to make here is that you are not *required* to use DI for your own services. However, you should at least have an understanding about how DI works in a Razor Pages application, so you can customize framework services as needed.

By the end of this chapter, you will understand what dependency injection is, how it is managed within a Razor Pages application, and the benefits of using DI for your own services. You will also have a clearer idea of what a service is in the context of a Razor Pages application. You will have created some services and registered them with the dependency injection container, so they are available throughout your application.

Services need to have their lifetime managed by the DI container, so your application doesn't run out of memory, for example. Neither should you instantiate services unnecessarily, especially if only one single instance is required by the application. In this chapter, you will learn the different lifetimes available and how to choose the correct one when registering your services.

7.1 The reason for dependency injection

Before we look at the fundamentals of dependency injection in ASP.NET Core, we need to be clear about the nature of the problem it solves. That